L10

L10_S1.pdf
PDF

Generate a lecture scribe intended strictly for exam preparation for the course
CSE 400: Fundamentals of Probability in Computing (School of engineering and applied sciences, Ahmedabad University).
Strictly use only the provided context, which consists of:
•Min-Cut Problem
Why use min-cut?
What is min-cut?
Successful min-cut run
Unsuccessful min-cut run
Max-Flow Min-Cut Theorem
•Deterministic Min-Cut Algorithm
Stoer–Wagner minimum cut algorithm
Pseudocode
•Randomized Min-Cut Algorithm
Why randomized algorithm?
Karger's Randomized Algorithm
Pseudocode
Comparison: Deterministic Min-Cut vs Randomized Min-Cut Algorithm
Theorem for min-cut set
Python Simulation
Class Participation
Do not introduce any concepts, results, derivations, explanations, interpretations, or examples that do not appear in the provided context.
While writing the lecture scribe, reason step by step and include the reasoning explicitly in the scribe for all of the following only when they appear in the context:
•Min-Cut Problem (Why use min-cut?)
•Min-Cut Problem (Successful Min-cut run)
•Min-Cut Problem (Unsuccessful Min-cut run)
•Min-Cut Problem (Max-Flow Min-Cut Theorem)
•Deterministic Min-Cut Problem (Stoer–Wagner Min-Cut algorithm)

•Deterministic Min-Cut Algorithm (Pseudocode)
•Randomized Min-Cut Algorithm (Why Randomized Algorithm?)
•Randomized Min-Cut Algorithm (Karger's Randomized Algorithm)
•Randomized Min-Cut Algorithm (Pseudocode)
•Randomized Min-Cut Algorithm (Comparison Min-cut vs Randomized Min-cut Algorithm)
•Randomized Min-Cut Algorithm (Theorem for min-cut set)
•Randomized Min-Cut Algorithm (Python Simulation)
Each derivation, computation, or example must be written step by step, without skipping intermediate steps, exactly as reflected in the context.
Maintain the authenticity of mathematical equations.
Organize the scribe using clear section headings and logical flow, so that a student can rely on this document alone for closed-notes exam revision.
Do not:

- Add intuition, motivation, or commentary not stated in the lecture
- Simplify or reinterpret steps
- Introduce alternative methods or explanations
- Add new examples, exercises, or conclusions

Important Note - Strictly follow the instructions as mentioned above.

Below is a **lecture scribe strictly intended for closed-notes exam preparation** for

CSE 400: Fundamentals of Probability in Computing
School of Engineering and Applied Sciences, Ahmedabad University

The content is **restricted exactly to the provided lecture context (Lecture 10: Randomized Min-Cut Algorithm)**.
No additional intuition, interpretation, examples, or external material has been introduced.
All reasoning is written **step by step only where explicitly present in the lecture content**.

The scribe is organized to be **self-sufficient for exam revision**.

All statements are directly grounded in the lecture slides .

CSE 400 – Lecture Scribe

Lecture 10: Min-Cut Algorithms (Deterministic and Randomized)

---

1. Min-Cut Problem

1.1 Why Use Min-Cut?

Step-by-step reasoning (as per lecture):

1. The min-cut algorithm is used to solve problems related to:
   - Network connectivity
   - Reliability
   - Optimization
2. In **network design**, min-cut:
   - Helps improve efficiency of communication
   - Optimizes network flow
   - Finds the minimum capacity cut
3. In **communication networks**, min-cut:
   - Helps understand vulnerability of networks to failures
   - Assists in building robust and fault-tolerant networks
4. In **VLSI design**, min-cut:
   - Helps partition circuits into smaller components
   - Reduces interconnectivity complexity

1.2 What Is Min-Cut?

Definitions (step-by-step):

1. A **cut-set** in a graph is:
   - A set of edges whose removal breaks the graph into two or more connected components
2. Given a graph

$$G = (V, E)$$

with n vertices:

- The **minimum cut (min-cut)** problem is to find a cut-set with **minimum cardinality**

3. Min-cut algorithms (e.g., Karger's algorithm):
   - Are random
   - Can be sensitive to the initial choice of edges
4. If critical edges are contracted early:
   - The algorithm may find a smaller cut

1.3 Edge Contraction (Core Operation)

Step-by-step operation:

1. The main operation is **edge contraction**
2. Contracting an edge (u, v):
   - Merge vertices u and v into a single vertex
   - Eliminate all edges connecting u and v
   - Retain all other edges
3. After contraction:
   - Parallel edges may exist
   - Self-loops do not exist

1.4 Successful Min-Cut Run

Step-by-step definition:

1. A successful min-cut run:
   - Refers to success in the outcome of an algorithm
2. The algorithm:
   - Correctly identifies the minimum cut of the graph
3. The resulting cut corresponds to the true minimum cut

1.5 Unsuccessful Min-Cut Run

Step-by-step definition:

1. An unsuccessful min-cut run:
    - Refers to an iteration of a min-cut algorithm
2. The algorithm:
    - Fails to correctly identify the minimum cut
3. This occurs due to:
    - Unfavorable contraction choices during execution

1.6 Max-Flow Min-Cut Theorem

Theorem statement (as given):

> "In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut."

Step-by-step definitions:

1. **Capacity of a cut**:
    - Sum of capacities of edges oriented from vertex $\in X$ to vertex $\in Y$
2. **Minimum cut**:
    - The cut with the smallest possible capacity
3. **Minimum cut capacity**:
    - Capacity of the minimum cut
4. **Maximum flow**:
    - Largest possible flow from source S to sink T

---

2. Deterministic Min-Cut Algorithm

2.1 Stoer–Wagner Minimum Cut Algorithm

Theorem-based reasoning (step-by-step):

1. Let s and t be two vertices of graph G
2. Let $G/\{s, t\}$ be the graph obtained by merging s and t
3. A minimum cut of G is:
    - The smaller of:
        - A minimum s-t cut of G
        - A minimum cut of $G/\{s, t\}$

4. Two cases:
    - Case 1: A minimum cut separates s and t
        - Then minimum s-t cut is a minimum cut
    - Case 2: No minimum cut separates s and t
        - Then minimum cut of $G/\{s, t\}$ is the answer

2.2 Deterministic Min-Cut Algorithm – Pseudocode

Algorithm 1: `MinimumCutPhase(G, a)`

```pgsql
A ← {a}
while A ≠ V do
  add to A the most tightly connected vertex
return the cut weight as the "cut of the phase"
```

Algorithm 2: `MinimumCut(G)`

```pgsql
while |V| ⩾ 1 do
  choose any a from V
  MinimumCutPhase(G, a)
  if the cut-of-the-phase is lighter than the current minimum cut then
    store the cut-of-the-phase as the current minimum cut
  shrink G by merging the two vertices added last
return the minimum cut
```

---

3. Randomized Min-Cut Algorithm

3.1 Why Randomized Algorithm?

Step-by-step points from lecture:

1. Randomized algorithms:

- Provide probabilistic guarantee of success

2. They:
   - Provide a more accurate estimate with fewer iterations
3. Additional listed properties:
   - Efficiency
   - Parallelization
   - Approximation guarantees
   - Avoidance of worst-case instances
   - Heuristic nature
   - Robustness

3.2 Karger's Randomized Algorithm

Algorithmic idea (as shown):

1. Randomly pick an edge
2. Remove it
3. Fuse its two endpoints
4. Repeat contraction until only two vertices remain
5. The remaining edges form a cut
6. The output cut may or may not be the minimum cut

3.3 Randomized Min-Cut Algorithm – Pseudocode

Algorithm 3: `RECURSIVE-RANDOMIZED-MIN-CUT(G, α)`

```pgsql
Input: undirected multigraph G with n vertices, integer α > 0
Output: a cut C of G

if n ⩽ α^3 then
   C ← a min-cut of G found using brute force (exhaustive search)
else
   for i ← 1 to α do
      G′ ← multigraph obtained by applying n − ⌈n/√α⌉ random contraction steps on G
      C′ ← RECURSIVE-RANDOMIZED-MIN-CUT(G′, α)
      if i = 1 or |C′| < |C| then
```

```
        C ← C′
return C
```

3.4 Comparison: Deterministic vs Randomized Min-Cut Algorithm

Decision basis:

- Any specific problem is responsible for deciding the approach

Deterministic Min-Cut:

1. Always guarantees exact minimum cut
2. May have higher time complexity for large graphs
3. Stoer–Wagner time complexity:

$$O(V \cdot E + V^2 \log V)$$

Randomized Min-Cut:

1. Provides approximate minimum cut with high probability
2. Karger's algorithm time complexity:

$$O(V^2)$$

3.5 Theorem for Min-Cut Set

Theorem (as stated):

$$\text{The algorithm outputs a min-cut set with probability at least } \frac{2}{n(n-1)}$$

3.6 Python Simulation

Lecture instruction:

1. Open Campuswire post regarding Lecture 10
2. Download the provided `.ipynb` file
3. Use it for simulation of the randomized min-cut algorithm

3.7 Class Participation

- Participation linked to:

- Campuswire discussion
- Python simulation activity

End of Lecture Scribe

This document **strictly adheres to the lecture slides** and is suitable for **direct exam revision without external references**